

A New Paradigm: Greedy Algorithms

The Core Philosophy

- Divide and Conquer breaks problems into smaller, independent subproblems.
- A **Greedy Algorithm** builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit.
- **The Rule:** Make the locally optimal choice at each stage with the hope that these local choices will lead to a global optimum.
- **No Regrets:** Once a greedy algorithm makes a choice, it *never* reconsiders it.

Two Key Ingredients for Success

- ① **Greedy Choice Property:** A global optimum can be arrived at by selecting a local optimum.
- ② **Optimal Substructure:** An optimal solution to the problem contains optimal solutions to the subproblems.

The Intuition: The Coin Change Problem

The Scenario

- You are a shopkeeper. You need to give a customer *Rs.38* in change.
- You have an infinite supply of coins: 20, 10, 5, 2, and 1.
- **Objective:** Minimize the total *number* of coins handed to the customer.

The Greedy Strategy

- **Rule:** Always pick the largest denomination coin that is less than or equal to the remaining target amount.
- **Example:** Make *Rs.38*.
 - Pick 20 (Remaining: *Rs.18*)
 - Pick 10 (Remaining: *Rs.8*)
 - Pick 5 (Remaining: *Rs.3*)
 - Pick 2 (Remaining: *Rs.1*)
 - Pick 1 (Remaining: *Rs.0*)
- **Result:** 5 coins. This is strictly optimal for the standard denomination system!

The Trap: When Greedy Fails

Changing the Rules

- What if we invent a new fictional currency system?
- **New Denominations:** $\{Rs.1, Rs.15, Rs.20\}$
- **New Target:** Make $Rs.30$ using the minimum number of coins.

Greedy vs. Optimal

- **The Greedy Path:**
 - Pick $Rs.20$ (Remaining: $Rs.10$)
 - Pick ten $Rs.1$ coins (Remaining: $Rs.0$)
- Greedy yields **11 coins** ($1 + 10$).
- **The Optimal Path:** $Rs.15 + Rs.15 = Rs.30$. This yields **2 coins!**

The Takeaway

- A greedy algorithm is only correct if we can mathematically *prove* the Greedy Choice Property holds for that specific problem structure.

Formalizing Greedy: Activity Selection

The Scenario

- You have a single resource (e.g., a lecture hall) and a set of n proposed activities $S = \{a_1, a_2, \dots, a_n\}$.
- Each activity a_i has a start time s_i and a finish time f_i , where $0 \leq s_i < f_i < \infty$.
- Two activities a_i and a_j are **compatible** if their intervals do not overlap (i.e., $s_i \geq f_j$ or $s_j \geq f_i$).
- **Objective:** Select a maximum-size subset of mutually compatible activities.

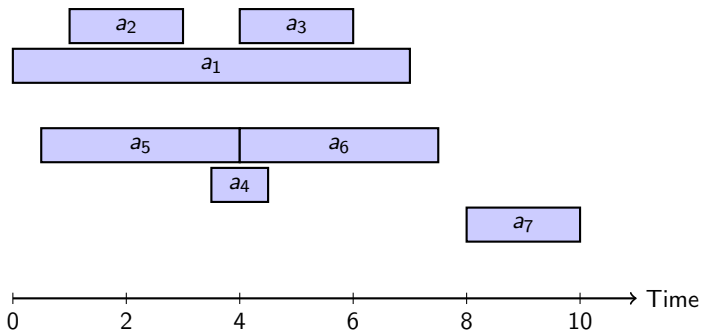
The Search for a Heuristic

- We need a rule to greedily pick the "best" next activity. What should our metric be?
- Let's test a few intuitive strategies visually.

Activity Selection: The Challenge

Visualizing a Pool of Activities

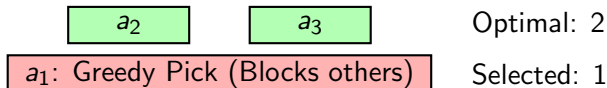
- We must select a subset such that **no two selected rectangles overlap**.
- Which one should we pick first?



Activity Selection: Failed Heuristics

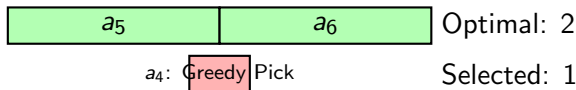
Idea 1: Pick the Earliest Start Time

- *Logic*: Start using the room as soon as possible.
- *Counterexample from pool* (a_1, a_2, a_3): a_1 starts first, but it is massive and blocks many smaller ones.



Idea 2: Pick the Shortest Duration

- *Logic*: Short activities consume the least resource time.
- *Counterexample from pool* (a_4, a_5, a_6): a_4 is shortest, but it bridges a crucial gap, blocking two compatible activities.



Activity Selection: The Optimal Strategy

The Winning Heuristic: Earliest Finish Time

- *Logic:* The resource becomes available for the next activity as early as possible, maximizing the remaining time available.

The Algorithm

- 1 **Sort** the activities by their finish times in monotonically increasing order: $f_1 \leq f_2 \leq \dots \leq f_n$.
- 2 Select the first activity corresponding to f_1 .
- 3 Iterate through the sorted list. If an activity starts *after* the previously selected activity finishes, select it.

Complexity

- Sorting takes $\mathcal{O}(n \log n)$.
- The greedy scan takes $\mathcal{O}(n)$.
- **Total Time Complexity:** $\mathcal{O}(n \log n)$.

Activity Selection: Pseudocode

Input: Arrays s and f of size n , representing start and finish times

Output: A set A of mutually compatible activities

Sort the activities such that $f_1 \leq f_2 \leq \dots \leq f_n$;

$A \leftarrow \{a_1\}$; // Always greedily select the first activity

$k \leftarrow 1$; // Index of the most recently selected activity

for $i \leftarrow 2$ **to** n **do**

if $s_i \geq f_k$ **then**

$A \leftarrow A \cup \{a_i\}$; // Activity i is compatible

$k \leftarrow i$; // Update the finish time boundary

end

end

return A ;

Activity Selection: Complexity Analysis

Time Complexity Breakdown

- **Sorting Phase:** Sorting the n activities by their finish times is the dominant operation. Using an optimal comparison-based sort, this strictly bounds to $\mathcal{O}(n \log n)$ time.
- **Greedy Scan (The For-Loop):** The algorithm makes a single, sequential pass through the array from index 2 to n .
- Inside the loop, checking the condition $(s_i \geq f_k)$ and updating variables executes in $\mathcal{O}(1)$ time. Thus, the entire scan processes $n - 1$ elements in exactly $\mathcal{O}(n)$ time.

Overall Time Complexity

- $T(n) = \mathcal{O}(n \log n) + \mathcal{O}(n) = \mathcal{O}(n \log n)$.
- *Note:* If the input arrays are *already sorted* by finish time prior to execution, the algorithm operates in strictly $\mathcal{O}(n)$ time.

Proof of Correctness: "Greedy Stays Ahead"

- **The Core Idea:** We compare our greedy schedule to a hypothetical optimal schedule step-by-step.
- We will prove that at every single step, the greedy choice finishes *at least as early* as the corresponding optimal choice. Because it frees up the resource faster, the optimal solution can never "outpace" it.

Setting up the Comparison

- Let our greedy solution be $G = \{g_1, g_2, \dots, g_k\}$, ordered by finish time.
- Let an optimal solution be $O = \{o_1, o_2, \dots, o_m\}$, also ordered by finish time.
- Since O is optimal, we know $m \geq k$. Our goal is to prove $m = k$.

The Inductive Proof

Claim: For every index r (where $1 \leq r \leq k$), the finish time of the r -th activity in the greedy schedule is less than or equal to the finish time of the r -th activity in the optimal schedule. Mathematically: $f(g_r) \leq f(o_r)$.

Base Case ($r = 1$):

- The greedy algorithm explicitly selects the activity with the absolute earliest finish time in the entire set.
- Therefore, it is guaranteed that $f(g_1) \leq f(o_1)$. The claim holds for $r = 1$.

Inductive Step:

- Assume the statement holds for $r - 1$, meaning $f(g_{r-1}) \leq f(o_{r-1})$.
- In the optimal schedule O , activity o_r must start after o_{r-1} finishes. So, $s(o_r) \geq f(o_{r-1})$.
- By our assumption, $s(o_r) \geq f(o_{r-1}) \geq f(g_{r-1})$.
- This means o_r is strictly compatible with our greedy schedule up to g_{r-1} . It is an *available option* for the greedy algorithm.

Concluding the Proof

Finishing the Inductive Step

- Since o_r is available, the greedy algorithm could have picked it.
- However, the greedy algorithm looks at all available options and picks the one with the *earliest finish time*.
- Therefore, whatever it actually picks (g_r) must finish no later than o_r .
- Thus, $f(g_r) \leq f(o_r)$. The greedy algorithm "stays ahead."

Why does this prove optimality?

- We know $f(g_k) \leq f(o_k)$.
- Suppose the optimal schedule had more activities than the greedy one ($m > k$).
- Then there is an activity o_{k+1} in the optimal schedule that starts after o_k finishes: $s(o_{k+1}) \geq f(o_k)$.
- But since $f(o_k) \geq f(g_k)$, it means $s(o_{k+1}) \geq f(g_k)$.
- This means o_{k+1} is compatible with the end of the greedy schedule. But the greedy algorithm only stops when there are *no compatible activities left*.

Proof of Correctness: Optimal Substructure

What is Optimal Substructure?

- A problem has optimal substructure if an optimal solution to the entire problem contains within it optimal solutions to its subproblems.
- To prove this, we use a standard technique called the "**Cut and Paste**" argument (a proof by contradiction).

Setting up the Subproblems

- Let S be our set of activities, and let A be an optimal schedule for S .
- Suppose we pick some activity $a_k \in A$.
- Choosing a_k divides our original problem into two independent subproblems:
 - S_{before} : Activities that finish before a_k starts.
 - S_{after} : Activities that start after a_k finishes.
- Consequently, our optimal solution A is divided into A_{before} and A_{after} .

The "Cut and Paste" Contradiction

The Hypothesis

- We claim that A_{after} **must** be an optimal solution for the subproblem S_{after} .

The Contradiction

- Assume the opposite: A_{after} is *not* optimal for S_{after} .
- This means there exists some better, valid schedule A'_{after} for the remaining time, such that $|A'_{after}| > |A_{after}|$.
- If this were true, we could "cut" A_{after} out of our schedule and "paste" A'_{after} in its place.
- Our new total schedule would be: $A_{before} \cup \{a_k\} \cup A'_{after}$.
- The total number of activities would now be strictly greater than $|A|$.
- **Contradiction!** We originally defined A as the maximum possible optimal schedule for S . It is impossible to find a larger one.

Proof of Correctness: Optimal Substructure

Theorem: Let $S = \{a_1, a_2, \dots, a_n\}$ be the set of activities sorted by finish time ($f_1 \leq f_2 \leq \dots \leq f_n$). There exists an optimal solution that includes the greedy choice, a_1 .

Proof:

- Let A be a hypothetical optimal solution.
- Let a_k be the activity in A with the earliest finish time.
- **Case 1:** If $a_k = a_1$, the optimal solution contains our greedy choice. We are done.
- **Case 2:** If $a_k \neq a_1$, we construct a new solution:

$$A' = (A \setminus \{a_k\}) \cup \{a_1\}$$

- Because S is sorted, we know $f_1 \leq f_k$. Since a_k does not overlap with any other activity in A , replacing it with a_1 (which finishes even earlier) guarantees a_1 will not overlap with the rest of A .
- Therefore, A' is a valid schedule. Since $|A'| = |A|$, A' is also an optimal solution containing a_1 .

Proof of Correctness: Optimal Substructure

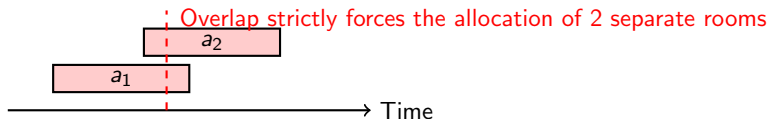
Completing the Proof by Induction

- We have proven the **Greedy Choice Property**: We can safely pick a_1 without sacrificing optimality.
- Once a_1 is chosen, we eliminate all activities that conflict with a_1 . Let the remaining compatible activities be S' .
- The remaining problem is to find the maximum number of compatible activities in S' .
- This is the *exact same problem*, just on a smaller set. Because the problem exhibits **Optimal Substructure**, we can apply the same greedy logic to S' , then S'' , and so on.
- **Conclusion**: The greedy algorithm is mathematically guaranteed to find a global optimum.

Interval Partitioning Problem

A Shift in Perspective

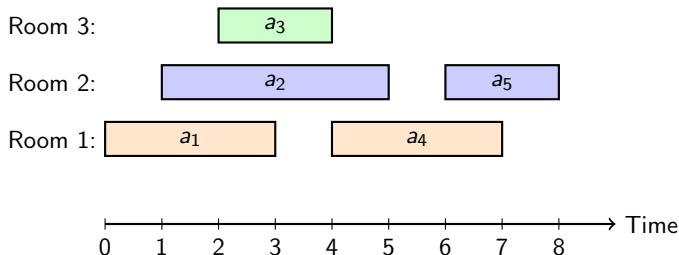
- In **Activity Selection**, we had *one* resource and wanted to schedule as many compatible activities as possible (leaving some behind).
- In **Interval Partitioning** (also known as the Classroom Scheduling Problem), we are obligated to schedule *every single activity*.
- We have an infinite supply of resources (e.g., identical lecture halls).
- **Input:** A set of n activities $S = \{a_1, a_2, \dots, a_n\}$. Each activity has a continuous time interval $[s_i, f_i)$.
- **Constraint:** Two overlapping activities cannot share the same room.
- **Objective:** Partition the activities into the **minimum possible number of rooms** such that no two activities in the same room overlap.



Visualizing the Objective

Packing Activities into Rooms

- We must assign each interval to a "lane" (room) without any collisions.
- Our goal is to pack these tightly to minimize wasted idle time between classes, thereby keeping the total number of lanes used as small as possible.



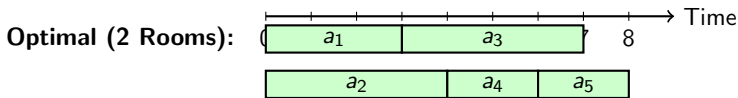
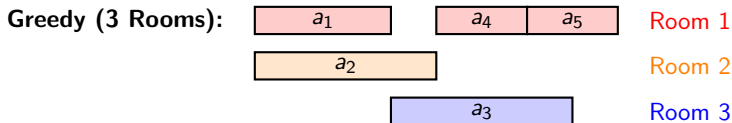
- In the example above, all 5 activities are successfully scheduled.
- The optimal algorithm must find this tightly packed configuration.

Strategy 1: Sequential Activity Selection

The Strategy: "Greedy by Earliest Finish Time, Repeatedly"

- *Idea*: Run our optimal single-room strategy (Earliest Finish Time) to pack Room 1 as full as possible. Then, run it again on the remaining activities for Room 2, and so on.
- *Why it fails*: Packing the first room optimally for *itself* can fragment the remaining activities, forcing them into inefficient configurations.

Counterexample



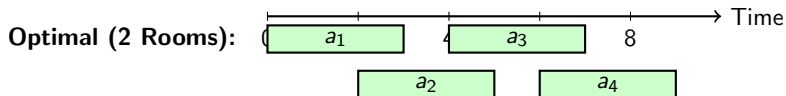
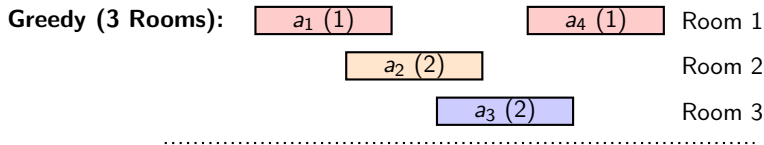
Strategy 2: Fewest Overlaps First

The Strategy: "Minimize Conflicts"

- *Idea*: Count how many other activities each activity overlaps with. Pick the one with the fewest overlaps first and assign it.
- *Why it fails*: Activities at the far ends of the day naturally have fewer overlaps. Scheduling them together can isolate the dense middle periods, forcing unnecessary room allocations.

Counterexample

a_1 and a_4 have **1 overlap**. a_2 and a_3 have **2 overlaps**.



The Winning Strategy: Earliest Start Time First

The Logic: Processing Chronologically

- Rather than looking at durations or finish times, we should process the activities exactly as they unfold in time.
- By sorting activities by their **start times**, we simulate the passage of a day.
- We only open a new room if, at the exact moment an activity starts, every single previously opened room is actively occupied.

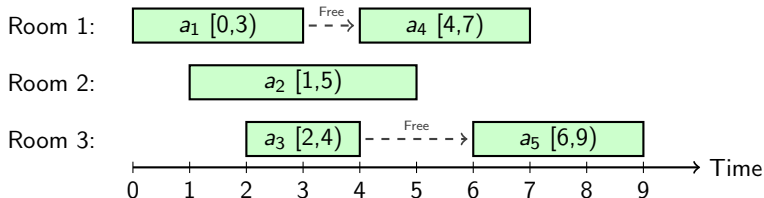
The Algorithm

- 1 **Sort** activities by start time: $s_1 \leq s_2 \leq \dots \leq s_n$.
- 2 For each activity a_i in the sorted list:
 - Check if any currently open room is free (i.e., its last activity finished at or before s_i).
 - If yes, assign a_i to that room.
 - If no (all rooms are busy), open a **new** room and assign a_i to it.

Visualizing the Execution

Step-by-Step Assignment

- Consider 5 activities sorted by start time: a_1, a_2, a_3, a_4, a_5 .

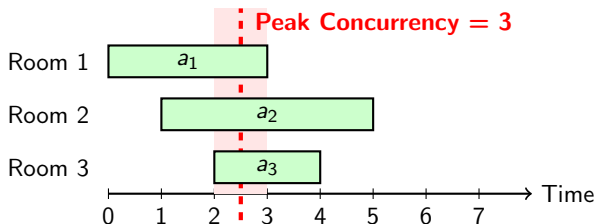


How the decisions were made:

- a_1, a_2, a_3 start at $t = 0, 1, 2$. Each forces a new room because previous rooms haven't finished yet.
- a_4 starts at $t = 4$. Room 1 became free at $t = 3$ and Room 3 became free at $t = 4$. We can reuse Room 1!
- a_5 starts at $t = 6$. Room 3 is currently free. We reuse Room 3!

The Result: The Absolute Minimum Number of Rooms

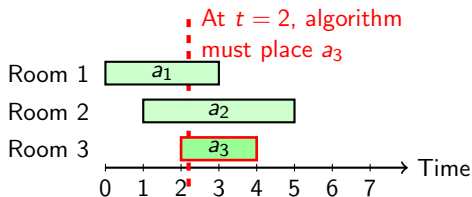
- What is the absolute minimum number of rooms physically required to cover *any* given schedule?
- Look at the maximum number of activities that overlap at any single exact moment. This peak concurrency or **depth** defines the absolute minimum limit.



- Because a_1 , a_2 , and a_3 all happen simultaneously between $t = 2$ and $t = 3$, no algorithm in the universe can use fewer than **3 rooms**.

Why It Never Wastes a Room

- How do we know the algorithm didn't open too many rooms by mistake?
- Let's look at the exact moment the algorithm was forced to open the **3rd room** (when scheduling a_3 at $t = 2$).



- As we sorted by start time, a_1 and a_2 *must* have started before a_3 .
- If Room 1 and Room 2 are unavailable at $t = 2$, their finish times *must* be strictly later than $t = 2$.
- Therefore, a_1 , a_2 , and a_3 all physically overlap at $t = 2$. By definition, the depth reached 3, making it physically impossible to use fewer than 3 rooms.

Formal Proof of Correctness: Definitions

The Setup

- Let d be the **depth** of the set of activities. Depth is defined as the maximum number of activities that overlap at any single point in time.
- Let O be the optimal (minimum possible) number of rooms required to schedule all activities.
- **Lemma 1 (Lower Bound):** $O \geq d$. It is fundamentally impossible to schedule d simultaneously overlapping activities in fewer than d rooms.
- Let k be the total number of rooms allocated by our Earliest Start Time greedy algorithm.

The Goal

- We know our algorithm produces a valid schedule, meaning $k \geq O$.
- We want to prove that the algorithm is perfectly optimal, which requires proving that $k = d$. Since $O \geq d$, we have $k \geq O \geq d$.

Formal Proof of Correctness: The Argument

Proof by Analysis of the Worst-Case Allocation

- Suppose the algorithm allocates exactly k rooms. Consider the exact moment it is forced to open the k -th room. Let it open this room to accommodate activity a_j .
- Activity a_j has a start time of s_j .
- Why did the algorithm open the k -th room? It only does this if rooms $1, 2, \dots, (k-1)$ were **all completely occupied** at time s_j .
- Let the activities occupying those $k-1$ rooms be $a_{i_1}, a_{i_2}, \dots, a_{i_{k-1}}$.

Establishing the Overlap

- Because our algorithm sorted all activities by start time, we know for a fact that every activity currently in those rooms started *before or exactly at* s_j . So, $s_i \leq s_j$ for all i in $\{i_1 \dots i_{k-1}\}$.
- Because these activities are actively occupying the rooms at time s_j , their finish times must be strictly *after* s_j . So, $f_i > s_j$.

Formal Proof of Correctness: The Conclusion

Connecting to Depth

- We have established that at time s_j , the following activities are all simultaneously happening:
 - ① The $k - 1$ activities in the previously opened rooms.
 - ② The new activity a_j .
- Therefore, exactly at time s_j , there are $(k - 1) + 1 = \mathbf{k}$ mutually overlapping activities.
- Since depth (d) is the *maximum* number of overlaps anywhere in the schedule, the overall depth must be at least the number of overlaps at time s_j .
- Thus, $\mathbf{d} \geq \mathbf{k}$.
- $k \geq 0 \geq d$ and $d \geq k \implies \mathbf{k} = \mathbf{0} = \mathbf{d}$.
- **Conclusion:** The algorithm allocates exactly d rooms. Since no algorithm can use fewer than d rooms, the greedy algorithm is strictly optimal. ■

Interval Partitioning: Pseudocode

Input: Array of n activities with start times s and finish times f

Output: Minimum number of rooms k , and their assignments

Sort the activities such that $s_1 \leq s_2 \leq \dots \leq s_n$;

Initialize a Min-Heap H ; // Stores finish times of active rooms

$k \leftarrow 0$; // Counter for total rooms allocated

for $i \leftarrow 1$ **to** n **do**

if H is not empty **and** $H.findMin() \leq s_i$ **then**

$H.extractMin()$; // Free the room that finished earliest

 Assign a_i to this reused room;

$H.insert(f_i)$; // Update the room's new finish time

else

$k \leftarrow k + 1$; // No rooms free, must open a new one

 Assign a_i to Room k ;

$H.insert(f_i)$; // Track the finish time of this new room

end

end

return k ;

Interval Partitioning: Complexity Analysis

Time Complexity Breakdown

- **Sorting Phase:** Sorting the n activities by their start times takes exactly $\mathcal{O}(n \log n)$ time.
- **Heap Operations Phase (The For-Loop):**
 - The algorithm iterates exactly n times.
 - Inside the loop, finding the minimum, extracting the minimum, and inserting a new finish time into the Min-Heap takes $\mathcal{O}(\log k)$ time, where k is the current number of rooms in the heap.
 - In the worst-case scenario, every activity requires a new room, meaning the heap grows to size n . Thus, heap operations take at most $\mathcal{O}(\log n)$ time per iteration.
 - The entire for-loop processes in $\mathcal{O}(n \log n)$ time.

Overall Complexity

- **Total Time Complexity:**

$$T(n) = \mathcal{O}(n \log n) + \mathcal{O}(n \log n) = \mathcal{O}(n \log n).$$
- **Space Complexity:** We need to store up to k finish times in the heap (where $k \leq n$). This requires $\mathcal{O}(n)$ auxiliary space.

A New Challenge: Scheduling with Deadlines

The Scenario: The Student's Dilemma

- You are a student facing a list of n assignments that must be submitted.
- **Single Resource Constraint:** You can only work on one assignment at a time.
- **No Preemption:** Once you start working on an assignment, you must work on it continuously until it is completely finished. You cannot pause it to start another.

The Parameters

- Each assignment i requires a known, continuous processing time of a_i days.
- Each assignment has a strict deadline d_i , measured in days from now ($t = 0$).
- **The Penalty:** For every day an assignment is overdue, a mark is deducted.

Formalizing the Objective: Maximum Lateness

Defining the Metrics

- A **schedule** is simply an ordering of these n assignments.
- Based on the schedule, we calculate the **Completion Time** (C_i) for each assignment i .
- The **Lateness** (L_i) of an assignment is the penalty incurred:

$$L_i = \max(0, C_i - d_i)$$

(If $C_i \leq d_i$, the assignment is on time, so the lateness penalty is 0.)

The Min-Max Objective

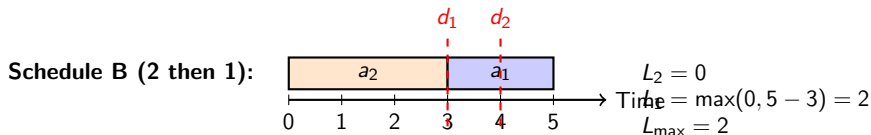
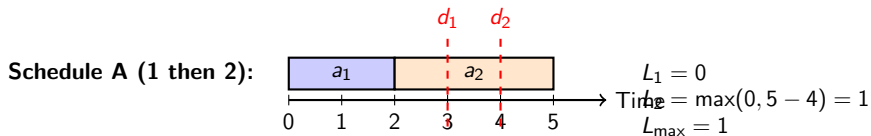
- Our goal is to protect our grade from catastrophic failure on any single assignment. We want to minimize the worst-case deduction.
- The **Maximum Lateness** of a schedule is:

$$L_{\max} = \max_{1 \leq i \leq n} (L_i)$$

- **Objective:** Find the schedule that minimizes $L_{\max}$.

Visualizing the Objective: Why Order Matters

- Assignment 1: Duration $a_1 = 2$, Deadline $d_1 = 3$
- Assignment 2: Duration $a_2 = 3$, Deadline $d_2 = 4$



- Schedule A achieves a lower maximum penalty (1 vs 2).
- Our goal is to find an algorithm that guarantees finding this optimal ordering for any number of assignments.

The Greedy Algorithm: Jackson's Rule

The Strategy: Earliest Deadline First (EDF)

- Proposed by J.R. Jackson in 1955, the algorithm is remarkably simple: prioritize urgency over size.

The Algorithm

- Sort** the n assignments in monotonically increasing order of their deadlines:

$$d_1 \leq d_2 \leq \dots \leq d_n$$

- Process** the assignments exactly in this sorted order, one after the other, with no idle time between them.

Key Characteristics

- The algorithm completely *ignores* the processing times (a_i). A 100-day assignment will be scheduled before a 1-day assignment if its deadline is earlier.
- Time Complexity:** The only computational work is the sorting step. Thus, the algorithm runs in strictly $\mathcal{O}(n \log n)$ time.

Proof of Correctness: The Exchange Argument

The Setup

- How do we prove this simple rule never fails? We use an **Exchange Argument**.
- Assume there exists an optimal schedule $\mathcal{O}$ that is *different* from our greedy EDF schedule $\mathcal{G}$.
- Because $\mathcal{O}$ is different, it must contain an **inversion**. An inversion occurs when an assignment i is processed immediately before an assignment j , but i has a later deadline ($d_i > d_j$).

Before (Inversion):

Assignment i	Assignment j
----------------	----------------

Conflict: $d_i > d_j$

$\longleftrightarrow$
 Total time used: $a_i + a_j$

After Swap:

Assignment j	Assignment i
----------------	----------------

$\longleftrightarrow$
 Total time used: $a_i + a_j$

Analyzing the Swap: The Mathematics

Let t be the exact time we start the first of the two assignments.

Before the Swap (Order: i then j)

- Completion time of i : $C_i = t + a_i$
- Completion time of j : $C_j = t + a_i + a_j$

After the Swap (Order: j then i)

- New completion of j : $C'_j = t + a_j$
- New completion of i : $C'_i = t + a_j + a_i$

Observation 1:

- Notice that $C'_i = C_j$. The swap only affects jobs i and j . All jobs after them in the schedule start at the exact same time as before.

Completing the Proof

Observation 2: What happens to the lateness?

- **For job j :** It now finishes earlier ($C'_j < C_j$). Therefore, its lateness strictly improves or remains zero ($L'_j \leq L_j$).
- **For job i :** It finishes later. Its new lateness is $L'_i = \max(0, C'_i - d_i)$.
- We know $C'_i = C_j$ (the new end time for i is the old end time for j).
- Because of the inversion, we know $d_i > d_j$.
- Therefore:

$$C'_i - d_i = C_j - d_i < C_j - d_j$$

- This means $L'_i \leq L_j \implies \max(L'_i, L'_j) \leq L_j \leq \max(L_i, L_j)$.

Conclusion

- Swapping i and j fixes the inversion without ever exceeding the old maximum lateness.
- By repeatedly swapping inversions, we eventually reach the EDF (greedy) schedule without ever worsening $L_{\max}$. ■

A Change in Objective: Total Lateness

- What if, instead of preventing a single catastrophic zero, we want to minimize the total marks lost across the entire semester?
- **Objective:** Minimize the sum of all lateness penalties:

$$\sum_{i=1}^n \max(0, C_i - d_i).$$

Does Jackson's Rule (EDF) still work?

- Let's look back at our Exchange Argument. We proved that swapping to fix an inversion guarantees:

$$\max(L'_i, L'_j) \leq \max(L_i, L_j)$$

- But what happens to the **sum** of the penalties? We need to compare $(L'_i + L'_j)$ against $(L_i + L_j)$.
- For the proof to work here, we will need $L'_j < L_i$ as well, which is not always true.

The Computational Reality of Total Lateness

The Tug-of-War

- Minimizing Total Lateness creates a fundamental conflict:
 - ① **Deadlines:** We want to do urgent tasks first (Jackson's Rule).
 - ② **Durations:** We want to do short tasks first to clear the queue and prevent bottlenecks (Shortest Processing Time).

NP-Hardness

- Because of this conflict, no simple greedy sorting rule can mathematically guarantee an optimal solution for all inputs.
- In fact, minimizing Total Lateness is formally classified as **NP-Hard**.

An Alternative: The Number of Late Assignments

Changing the Penalty Metric

- Since calculating the exact sum of days late is NP-hard, let's look at a different, highly practical metric.
- Suppose a late assignment just gets a flat zero, regardless of *how* late it is.
- Let $U_i = 0$ if assignment i is on time ($C_i \leq d_i$), and $U_i = 1$ if it is late ($C_i > d_i$).
- **New Objective:** Minimize $\sum U_i$ (or equivalently, maximize the number of on-time assignments).

The Return of Polynomial Time

- Unlike total lateness, this objective *can* be solved perfectly using a clever greedy algorithm in polynomial time.

Minimizing Number of Late Assignments

The Objective

- We want to maximize the number of on-time assignments.
- *Intuition:* We should start by following Jackson's Rule (Earliest Deadline First) to pack as many assignments as possible.

The Bottleneck

- Imagine we are adding assignments one by one in EDF order. Suddenly, we add assignment i , and our total time t exceeds its deadline d_i .
- We have officially "breached" a deadline. We cannot complete all assignments scheduled so far.
- **The Question:** We must drop exactly one assignment from our current set to get back on track. Which one should we eject to maximize our chances of completing future assignments? Can you prove your choice is optimal?

The Divisible Loot problem

The Scenario: The Explorer's Dilemma

- You are an explorer who has discovered a vault containing n different types of precious materials.
- **Capacity Constraint:** You only have a single bag that can hold a maximum total weight of W .
- **Divisible Goods:** Unlike solid artifacts, these materials are continuous substances like gold dust, silver powder, or rare spices. You can take any fractional amount of a material.

The Parameters

- Each material i has a total available physical weight of w_i .
- The entire pile of material i has a total monetary value of v_i .
- **The Choice:** You must decide what fraction x_i (where $0 \leq x_i \leq 1$) of each material to pack.

Formalizing the Objective: Maximizing Yield

Defining the Metrics

- A **solution** is an array of fractions $x_1, x_2, \dots, x_n$ representing how much of each material you take.
- If you take a fraction x_i of material i :
 - It consumes $x_i \cdot w_i$ of your bag's weight capacity.
 - It contributes $x_i \cdot v_i$ to your total wealth.

The Optimization Goal

- **The Strict Constraint:** The total weight of your selected fractions cannot break the bag:

$$\sum_{i=1}^n x_i \cdot w_i \leq W$$

- **Objective:** Find the combination of fractions that maximizes your total profit:

$$\max \sum_{i=1}^n x_i \cdot v_i$$

Designing a Greedy Choice: What to Prioritize?

Flawed Strategies

- **Option A: Highest Total Value First.** Taking the most valuable material might consume all your capacity with something extremely heavy, leaving no room for other good items.
- **Option B: Lowest Weight First.** Taking the lightest materials maximizes the *number* of items in your bag, but they might be practically worthless.

The Optimal Strategy: Value Density

- To maximize profit, we need to evaluate the "bang for your buck" of each material.
- We define the **Value Density (or unit value)** r_i of material i as its value-to-weight ratio:

$$r_i = \frac{v_i}{w_i}$$

- **Greedy Rule:** Always prioritize the material with the highest value per unit of weight.

The Algorithm: Sorting and Filling

Step-by-Step Process

- **Step 1: Calculate.** Compute the value density $r_i = v_i/w_i$ for all n materials.
- **Step 2: Sort.** Order the materials in monotonically decreasing order of their densities:

$$r_1 \geq r_2 \geq \cdots \geq r_n$$

- **Step 3: Pack.** Process the materials in this sorted order. Let W_{rem} be the remaining capacity of your bag (initially $W_{rem} = W$).
 - If the whole material fits ($w_i \leq W_{rem}$): Take all of it. Set $x_i = 1$ and update $W_{rem} = W_{rem} - w_i$.
 - If the whole material does *not* fit ($w_i > W_{rem}$): Take exactly the fraction that fills the bag. Set $x_i = \frac{W_{rem}}{w_i}$, update $W_{rem} = 0$, and **stop**.

A Numerical Example

The Setup: Bag Capacity $W = 50$

Material (i)	Value (v_i)	Weight (w_i)	Density (r_i)
A (Gold Dust)	100	20	5
B (Saffron)	60	10	6
C (Silver Powder)	120	30	4

Execution Steps

- **Sort by Density:** We process the materials in order B, A, C.
- **Pack Material B:** Fits completely ($10 \leq 50$). Take 100% ($x_B = 1$).
 - Value gained: 60. Remaining capacity: $50 - 10 = 40$.
- **Pack Material A:** Fits completely ($20 \leq 40$). Take 100% ($x_A = 1$).
 - Value gained: 100. Remaining capacity: $40 - 20 = 20$.
- **Pack Material C:** Bag is too small ($30 > 20$). Take a fraction.
 - Fraction taken: $x_C = \frac{20}{30} = \frac{2}{3}$.
 - Value gained: $\frac{2}{3} \cdot 120 = 80$. Remaining capacity: 0.

Result: Total optimal yield = $60 + 100 + 80 = 240$.

Analysis: Complexity and Correctness

Time Complexity

- Calculating ratios takes $\mathcal{O}(n)$ time.
- Sorting the items takes $\mathcal{O}(n \log n)$ time.
- The packing loop touches each item at most once, taking $\mathcal{O}(n)$ time.
- **Total Time:** The algorithm is dominated by the sorting step, making it $\mathcal{O}(n \log n)$.

Why does this always work? (Exchange Argument)

- Suppose an "optimal" solution $\mathcal{O}$ differs from our greedy solution $\mathcal{G}$.
- This means $\mathcal{O}$ must have taken some amount of a lower-density material instead of a higher-density material.
- **The Swap:** We can replace a tiny weight ϵ of the low-density material in $\mathcal{O}$ with weight ϵ of the available high-density material.
- The total weight remains identical, but the total value strictly increases—proving the greedy choice never fails! ■

A New Challenge: The Communicator's Dilemma

The Scenario: Massive Data Transmission

- You are tasked with transmitting a massive text file across a slow, expensive network.
- The file is written using a specific alphabet Σ (e.g., standard English letters).
- To transmit the file, every character must be encoded into binary (0s and 1s).

The Inefficiency of Fixed-Length Codes

- Standard encodings like ASCII use a **fixed length** (e.g., 8 bits) for every single character.
- If our alphabet has k distinct characters, a fixed-length code requires $\lceil \log_2 k \rceil$ bits per character.
- **The Challenge:** In real languages, frequencies are highly skewed. The letter 'e' might appear millions of times, while 'z' appears rarely. Paying the exact same 8-bit "cost" for 'e' as for 'z' wastes a massive amount of bandwidth.

The Trap: Variable-Length Codes and Ambiguity

The Intuitive Idea

- To save space, we should use **variable-length codes**: assign very short binary strings to frequent characters, and longer strings to rare characters.

The Decoding Challenge: The Ambiguity Trap

- Suppose we use the following variable-length encoding:

$$a = 0, \quad b = 1, \quad c = 01, \quad d = 10$$

- You receive the binary stream: 0101
- How do you decode it?
 - Is it 0, 1, 0, 1 $\implies$ **abab**?
 - Is it 01, 01 $\implies$ **cc**?
 - Is it 0, 10, 1 $\implies$ **adb**?
- Because codes overlap, the receiver cannot determine where one character ends and the next begins without complex look-ahead or delimiter bits (which ruin compression).

Formalizing the Solution Space: Prefix Trees

The Prefix Rule

- To ensure instantaneous, unambiguous decoding, we must strictly enforce the **Prefix Rule**: *No character's code can be a prefix of another character's code.*
- If 'a' is encoded as 0, no other character can start with 0.

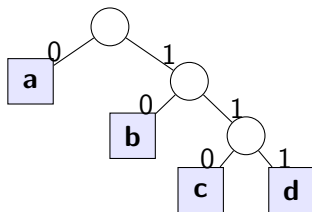
Defining the Optimization Objective

- A valid prefix code can be perfectly mapped to a binary tree, where characters are strictly at the **leaves**. Left branches = 0, Right branches = 1.
- Let $f(c)$ be the frequency of character c in the file.
- Let $d_T(c)$ be the depth of character c in a proposed tree T (this is exactly the length of its binary code).
- **The Cost Function**: The total bits required to encode the entire file is:

$$B(T) = \sum_{c \in \Sigma} f(c) \cdot d_T(c)$$

Visualizing a Prefix Code: The Tree Structure

By restricting characters exclusively to the **leaf nodes**, we mathematically guarantee that no code can be a prefix of another.



- **Codes derived from paths:** $a = 0$, $b = 10$, $c = 110$, $d = 111$.
- **Decoding is now unambiguous:** Try to decode 0110100
- Start at root. Read '0' → hit leaf **a**. (Return to root)
- Read '1', then '1', then '0' → hit leaf **c**. (Return to root)
- Read '1', then '0' → hit leaf **b**. (Return to root)
- Read '0' → hit leaf **a**.
- Exact decoded message: **acba**. No look-ahead required!

Designing a Greedy Choice: Top-Down vs. Bottom-Up

- To minimize the total cost $B(T) = \sum f(c) \cdot d_T(c)$, characters with the *highest* frequencies should have the shortest depths (closest to the root).
- Conversely, characters with the *lowest* frequencies can afford to have the longest depths (deepest in the tree).

The Greedy Insight (Huffman, 1952)

- Building the tree from the top down (trying to split the alphabet into high/low frequency halves) is notoriously difficult to optimize.
- **Huffman's Method:** Build the tree **bottom-up**. The two characters with the absolute *lowest* frequencies must sit at the very bottom of the optimal tree, acting as sibling leaves.
- If we merge these two least-frequent characters into a single "combined" character, we can reduce the problem size and repeat.

Huffman's Algorithm: The Step-by-Step Logic

Initialization

- Start with a set of n leaves, one for each character $c \in \Sigma$, each with a weight equal to its frequency $f(c)$.
- Insert all n leaves into a Priority Queue (a Min-Heap) ordered by frequency.

The Merging Loop

- While there is more than one node in the Priority Queue:
 - ① **Extract:** Remove the two nodes x and y with the lowest frequencies.
 - ② **Merge:** Create a new internal node z with x as its left child and y as its right child.
 - ③ **Update Weight:** Set the frequency of z to the sum of its children:

$$f(z) = f(x) + f(y)$$

- ④ **Insert:** Put the new combined node z back into the Priority Queue.

Termination

- When exactly one node remains in the queue, the algorithm terminates. This final node is the root of the optimal prefix tree.

Executing Huffman: A Step-by-Step Example

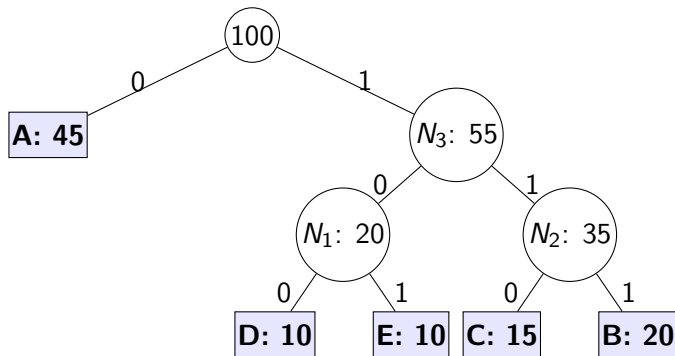
Initial State

- **Alphabet & Frequencies:** A: 45, B: 20, C: 15, D: 10, E: 10
- **Priority Queue (Q):** $\{D : 10, E : 10, C : 15, B : 20, A : 45\}$

The Merging Process (Always extract two minimums)

- **Step 1:** Merge D and E . Create $N_1 = 20$.
Q becomes: $\{C : 15, B : 20, N_1 : 20, A : 45\}$
- **Step 2:** Merge C and B . Create $N_2 = 35$.
Q becomes: $\{N_1 : 20, N_2 : 35, A : 45\}$
- **Step 3:** Merge N_1 and N_2 . Create $N_3 = 55$.
Q becomes: $\{A : 45, N_3 : 55\}$
- **Step 4:** Merge A and N_3 . Create Root = 100. Queue has 1 item.
Stop.

Executing Huffman: A Step-by-Step Example



Analysis of Huffman's Algorithm: Time Complexity

Initialization Phase

- We start with an alphabet of n distinct characters.
- Creating a leaf node for each character and building the initial Priority Queue (typically implemented as a Min-Heap) takes $\mathcal{O}(n)$ time using a standard heapify operation.

The Merging Loop

- The main loop executes exactly $n - 1$ times. Every iteration merges two nodes into one, reducing the total number of independent trees by exactly one until only the root remains.
- In each iteration, we perform three priority queue operations:
 - Two extract-min operations to find the lowest frequencies:
 $2 \times \mathcal{O}(\log n)$
 - One insert operation to place the new combined node back into the heap: $\mathcal{O}(\log n)$
- Thus, a single iteration takes $\mathcal{O}(\log n)$ time.
- The total time spent in the loop is $(n - 1) \times \mathcal{O}(\log n) = \mathcal{O}(n \log n)$.

Proof of Optimality: The Greedy Claim

The Core Assumption

- Let x and y be the two characters in our alphabet with the absolute lowest frequencies.
- **Claim:** There exists an optimal prefix tree where x and y are sibling leaves at the maximum depth of the tree.

The Setup (Assume the opposite)

- Suppose we have a theoretical optimal tree T where this is *not* true.
- Let a and b be two sibling leaves that actually sit at the maximum depth in T .
- Because x and y are the lowest frequencies overall, we know:

$$f(x) \leq f(a) \quad \text{and} \quad f(y) \leq f(b)$$

- Because a and b are at the very bottom, their depths are greater than or equal to x and y :

$$d_T(a) \geq d_T(x) \quad \text{and} \quad d_T(b) \geq d_T(y)$$

The Swap: Proving the Cost Never Increases

- What happens if we swap the position of character x with a to create a new tree T' ?
- The character (a) moves up to a shorter depth, cost decreases by $f(a) \cdot (d_T(a) - d_T(x))$.
- And the character (x) moves down to a longer depth. Cost decreases by $f(x) \cdot (d_T(x) - d_T(a)) = -f(x) \cdot (d_T(a) - d_T(x))$
- Total decrease in cost: $(f(a) - f(x)) \cdot (d_T(a) - d_T(x))$
- Since $f(a) \geq f(x)$ and $d_T(a) \geq d_T(x)$, both terms are positive. Thus, $\Delta \text{Cost} \geq 0$.
- This means swapping them strictly improves the tree (or keeps the cost exactly the same).
- Repeating this swap for y and b yields a new tree T'' where x and y are siblings at the bottom. Since T was optimal, T'' must also be optimal.
- **Result:** The greedy choice is always safe! ■

Optimal Substructure

Defining the Subproblem

- Let Σ be our original alphabet. Let x and y be the two characters with the lowest frequencies.
- Let Σ' be a smaller alphabet where x and y are removed, and a new combined character z is added, with frequency $f(z) = f(x) + f(y)$.

The Unmerging Step

- Suppose we recursively run our algorithm and find the strictly optimal prefix tree T' for the smaller alphabet Σ' .
- We reconstruct a tree T for the original alphabet Σ by taking T' and "unmerging" leaf z —replacing it with an internal node that has x and y as its children.
- **Claim:** The reconstructed tree T is guaranteed to be the optimal tree for the original alphabet Σ .

Analyzing the Cost of Unmerging

How does the cost change?

- In the reconstructed tree T , the depth of x and y is exactly one level deeper than z was in T' . So, $d_T(x) = d_T(y) = d_{T'}(z) + 1$.
- The cost contribution of x and y in T is:

$$\begin{aligned} & f(x) \cdot d_T(x) + f(y) \cdot d_T(y) \\ &= f(x)(d_{T'}(z) + 1) + f(y)(d_{T'}(z) + 1) \\ &= (f(x) + f(y)) \cdot d_{T'}(z) + f(x) + f(y) \end{aligned}$$

- Since $f(z) = f(x) + f(y)$, we can rewrite the total cost of tree T as:

$$B(T) = B(T') + f(x) + f(y)$$

Completing the Proof: The Contradiction

Assume T is NOT optimal

- Suppose there is some other tree T^* for Σ that is strictly better ($B(T^*) < B(T)$).
- We can assume without loss of generality that x and y are siblings in T^* .
- If we take T^* and merge x and y into z , we get a valid tree $T^{*'} for the smaller alphabet Σ' .$
- Reversing our cost formula: $B(T^{*'}) = B(T^*) - f(x) - f(y)$.
- If $B(T^*) < B(T)$, then subtracting $(f(x) + f(y))$ from both sides means:

$$B(T^{*'}) < B(T')$$

- **Contradiction!** We stated that T' was the absolute optimal tree for Σ' . We cannot find a better tree $T^{*'}$. Therefore, our assumption is false, and T must be optimal! ■